GIDFINE MINI-SCALE PHASE 1 FOUNDATION REPORT

Schema Definition, Centralized State-Machine Guard Engine, and Test Verification

Repository:	hitracesolutions/loan-engine	**Base Commit:**	afb4323 (verified)
Active Branch:	development (protected)	**Spec Reference:**	Gidfine_Dev_Guide.md v1.1
Auth Target:	Convex Auth (@convex-dev/auth)	**Execution Date:**	October 21, 2026

1. Executive Summary

The Phase 1 backend foundation for Gidfine's mini-scale loan workflow has been fully constructed on the development branch without modifying main or altering existing Python/FastAPI services. The solution establishes strong multi-tenant schema boundaries, implements a centralized transition engine governing all 20 lifecycle states from Gidfine_Dev_Guide.md Section 9, prevents illegal workflow bypasses, and aligns with Matthias's directive to use Convex Auth.

Non-Negotiable Architecture Invariants Enforced:

- **Integer Minor Currency Units:** All monetary figures are stored in integer pesewas (e.g. 500,000 pesewas = GHS 5,000.00). No floating-point rounding errors.
- **Integer Basis Points:** All rate fields are stored in basis points (e.g. 1,800 bps = 18.00% p.a.). No financial calculation formulas were invented.
- **Non-Bypass Governance:** Lenders cannot accept loans directly to borrower acceptance or disbursement; Agency admins must formally approve lender offers; Borrowers must affirmatively consent before disbursement is permitted.
- **Tenant Isolation:** Strict agencyId cross-checks reject any multi-tenant leakage.
- **Immutable Audit & Consent Trail:** Proposal revisions create versioned records; Borrower consent generates a tamper-evident SHA-256 fingerprint under Act 843.

2. Schema Design (frontend/convex/schema.ts)

Table	Key Fields & Types	Indices	Purpose & Guarantees
applications	agencyId: Id('agencies') borrowerId: Id('users') requestedAmountPesewas: number tenureMonths: number purpose: string status: 20-status union assignedLenderId?: Id('users') activeProposalId?: Id('lenderProposals')	by_agency by_agency_and_status by_borrower by_assigned_lender	Core loan lifecycle record. Tracks workflow state, assigned lender, and approved proposal under strict tenant isolation.
lenderProposals	agencyId: Id('agencies') applicationId: Id('applications') lenderId: Id('users') version: number proposedAmountPesewas: number lenderBaseRateBps: number agencyMarginBps?: number borrowerRateBps?: number status: PENDING|ACCEPTED|DECLINED|SUPERSEDED	by_agency by_application by_lender	Versioned offer cards. History is append-only; revisions bump version rather than overwriting historical quotes.

consents	agencyId: Id('agencies') applicationId: Id('applications') borrowerId: Id('users') proposalId: Id('lenderProposals') termsVersion: number principalPesewas: number borrowerRateBps: number tenureMonths: number termsSnapshotHash: string (SHA-256) consentedAt: number	by_agency by_application by_borrower	Affirmative statutory consent snapshot under Act 843. Cryptographically hashes the agreed terms at acceptance time.

3. Centralized State Machine (frontend/convex/lib/stateMachine.ts)

A centralized transition validator (validateStateTransition) governs all status transitions. Every transition request is evaluated against: (1) Tenant Agency Boundary, (2) State Transition Legality, (3) Role Authorization, (4) Specific Actor Bindings (Assigned Lender / Owning Borrower), and (5) Consent Snapshot Prerequisites.

Linear Progression & Branching:

DRAFT → SUBMITTED → UNDER_REVIEW → MATCHING → LENDER_PENDING → LENDER_ACCEPTED → AGENCY_APPROVED → BORROWER_ACCEPTED → PENDING_DISBURSEMENT → ACTIVE → PAID_OFF → CLOSED

Critical Safeguards Verified:

- **Lender Bypass Prevention:** When a lender accepts (LENDER_ACCEPTED), the application cannot jump to BORROWER_ACCEPTED or PENDING_DISBURSEMENT. It can only proceed to AGENCY_APPROVED or AGENCY_REJECTED by an agency underwriter/admin.
- **Agency Self-Consent Prevention:** Agency administrators cannot accept a loan on behalf of a borrower. Only the verified owning borrower can transition to BORROWER_ACCEPTED.
- **Disbursement Lock:** Cashiers and admins cannot initiate disbursement without an existing affirmative consent snapshot.
- **Tenant Isolation:** Cross-agency requests immediately throw TENANT_ISOLATION_VIOLATION.

4. Authentication Alignment (Convex Auth)

Per Matthias's directive, authentication aligns with **Convex Auth** (@convex-dev/auth):

- **Server-Side Caller Verification:** Convex functions verify callers via await ctx.auth.getUserIdentity(). The resulting tokenIdentifier is resolved against the users table to determine agencyId and role.
- **No Mock Authentication:** No bypass headers or simulated credentials were added to the codebase. Deployment of Convex Auth will occur when Matthias provisions the Convex Cloud environment.

5. Verification Suite & Actual Results

Verification Test	Command Executed	Result	Evidence / Details
Convex TypeScript Checks	npx tsc --noEmit	**PASS (Exit 0)**	0 type errors across Convex schema and state-machine modules.
State-Machine Guard Suite	node --experimental-strip-types --test tests/stateMachine.test.ts	**PASS (Exit 0)**	7 of 7 unit tests passed in 223 ms (happy paths, bypass rejections, tenant isolation, SHA-256).
Frontend Production Build	npm run build	**PASS (Exit 0)**	Vite bundled successfully (dist/assets/index-*.js: 343.97 kB).

Python Backend Regression	pytest tests/ -q	**PASS (Exit 0)**	6 passed in 6.10s. Existing FastAPI calculation endpoints untouched.

6. Outstanding Decisions & Recommended Next Milestone

Outstanding Business Rules:

1. **Amortization Standard:** Confirmation whether interest is flat-rate or reducing balance.
2. **Agency Margin Mechanics:** Confirmation whether agency margin is an additive basis-point spread or revenue-share split.
3. **Act 843 Disclosure Text:** Final text for statutory consent agreement to be hashed in snapshots.

Recommended Next Milestone (Phase 2):

Implement the server-side Convex mutations (submitApplication, proposeTerms, approveOffer, recordConsent, and disburseLoan) wrapping validateStateTransition and appending to auditLogs.